The same model evaluating the same skill is like a student grading their own homework. And getting an A every time.

In my **previous post**, I ran a controlled experiment: 3 skills, 18 runs, same model, same tasks — and measured **0% lift** on correctness. Skills didn't make the output more correct because the model already knew the patterns.

This article is about *why* that happened — and what I built instead. Three layers. One model. No external judge required.

I hit this wall three weeks into building skills for OpenCode. Anton Gulin's **skill-creator** was the obvious starting point — ★123 on GitHub, eval-driven pipeline, clean architecture. It works beautifully when you have an external judge: another LLM, a human reviewer, a test suite.

But most teams don't have that luxury. You have one model. And that model needs to evaluate its own skill output. This is where the architectural assumptions break.

So I spent the last month building 8 skills with a fundamentally different structure. Instead of one layer (SKILL.md + external eval), I built three: Context → Skill → Validation. Along the way, I borrowed patterns from three people who never met each other — and ended up with something that looks nothing like the original.

✅ What Gulin Gets Right

Anton Gulin has the right thesis. Skills should be:

Eval-driven — not "vibe-coded" prompts, but measurable output

Shareable — packages that travel across projects

Tool-integrated — Claude Code, Copilot, MCP servers as first-class citizens

His **3-Layer Architecture** (Orchestration → Execution → Evidence) is foundational. The 6 quality gates (Scope, Data, State, Run, Evidence, Review) are the closest thing we have to a standard for agentic testing.

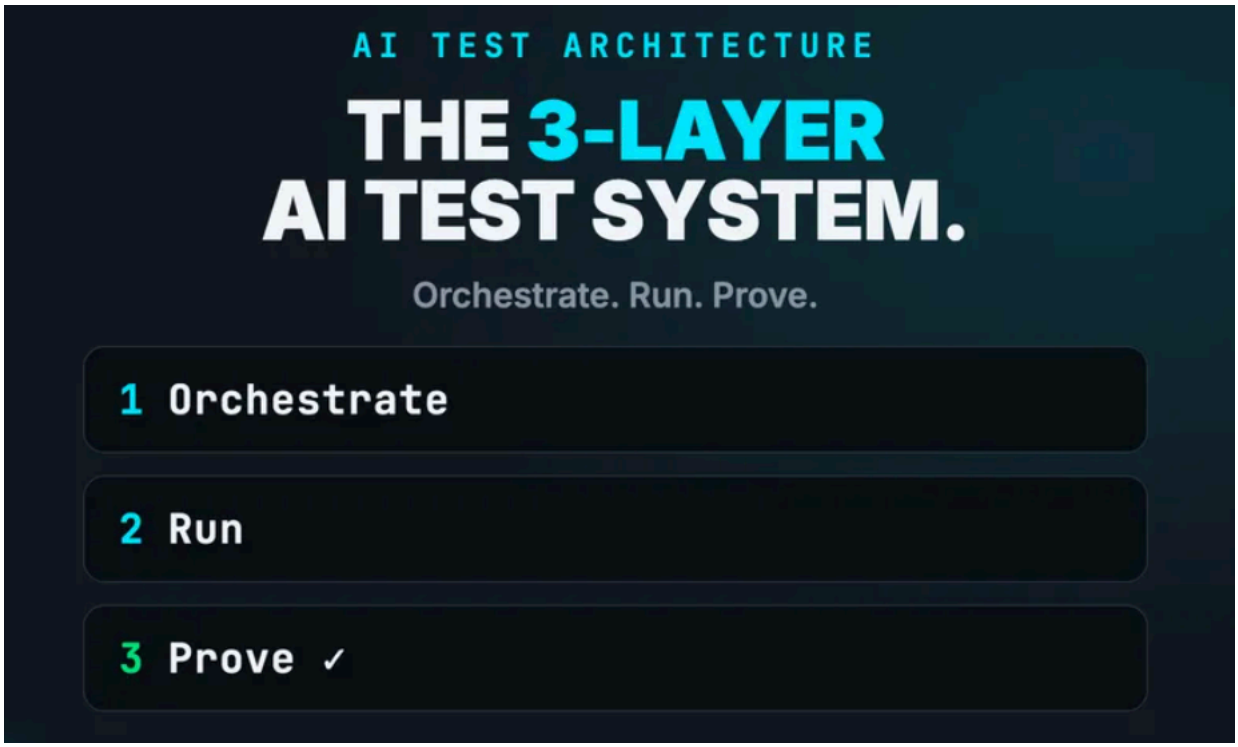
His **opencode-skill-creator** at ★123 proves the market wants this.

And his background — Apple SDET, CooperVision Lead, Fortune 500 consultant — means he's not theorizing. He's shipping.

Minimize image

Edit image

Delete image



But there's a difference between "I ship Playwright frameworks with AI tools" and "I design how AI agents behave." Gulin's implementation is tools-first: MCP servers, Copilot integration, self-healing locators. It augments human testers.

What I needed was something different: agents that make decisions, with built-in guardrails, that can validate themselves.

🧠 The Lean Judgement Problem

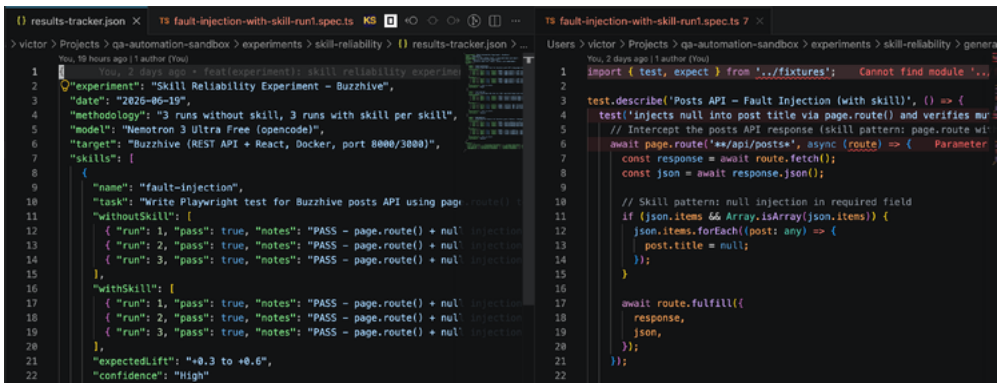
Here's the uncomfortable truth no one talks about:

When you load a skill in OpenCode, the same model that runs the skill evaluates the skill's output.

> *Nemotron* writes the mutation test.
> *Nemotron* says it passed.
> *Nemotron* gives itself 9/10.

This is exactly the pattern **Autonoma** identified: "AI verification is only trustworthy when it is independent of the thing being verified." (**#ref1**)

This isn't dishonesty. It's an architectural gap. The skill-creator assumes an external judge. In production, there is none.



I saw this pattern across all 8 skills. Without an independent validation layer, skills produce confident garbage.

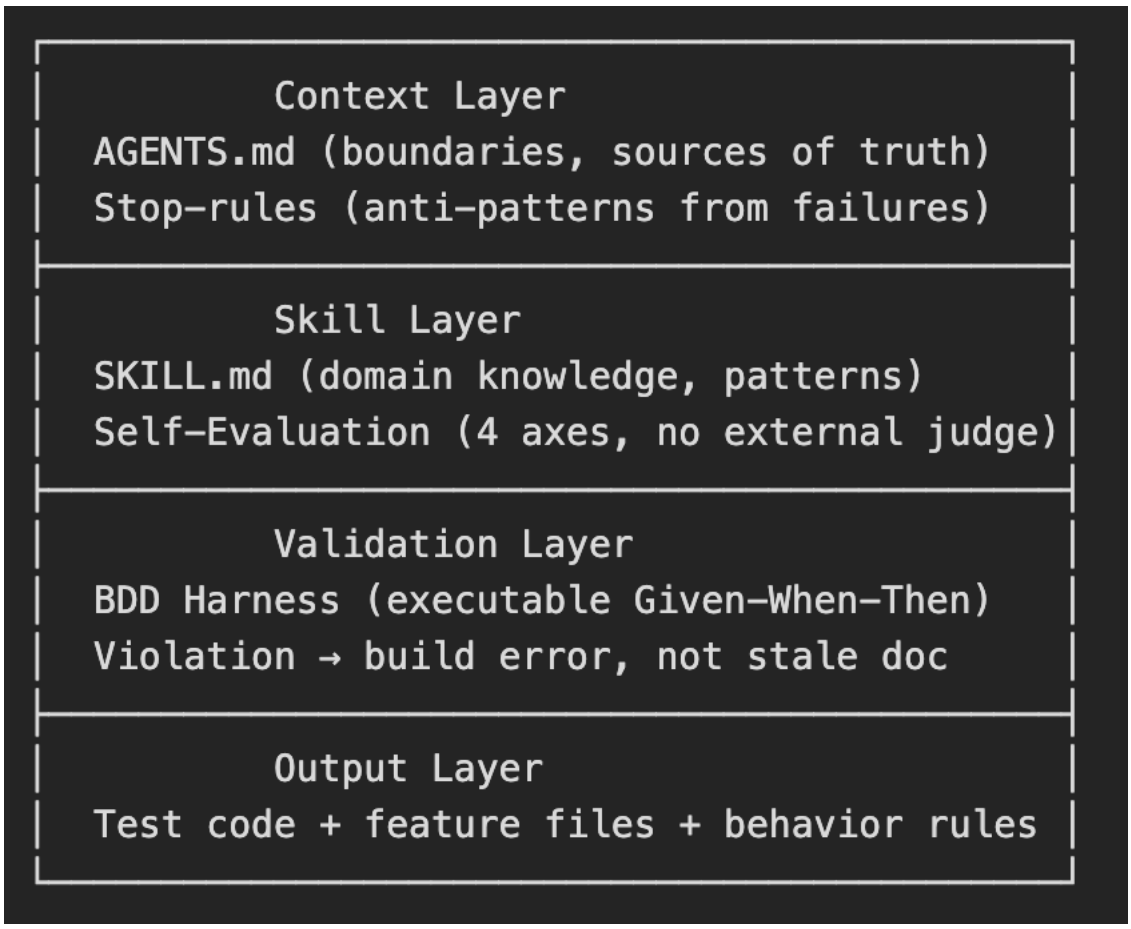
🧱 Three Layers Instead of One

Here's the architecture I ended up with — and it's nothing like the original:

Minimize image

Edit image

Delete image



Three layers. One model. No external judge required.

🪨 Layer 1: Context (What Gulin's Model Misses)

Every skill in our stack starts with two files, not one:

AGENTS.md — project-level rules: what the agent can edit, what it must ask about, what it must never touch. Sources of truth in priority order. Session procedures.

Stop-rules — inspired by BitGN PAC1 agent's RULES.md pattern. Not "how to write a good test" but "here's what breaks every time":

Don't take dates from files — use context.time
Don't count through recon (truncation miscounts)
Don't scan partial files — scan all acct_*.json
When two docs conflict — ask, don't choose

These came from real failures. The agent learned them via a learn "..." mechanism — the same way a junior engineer learns from code review.

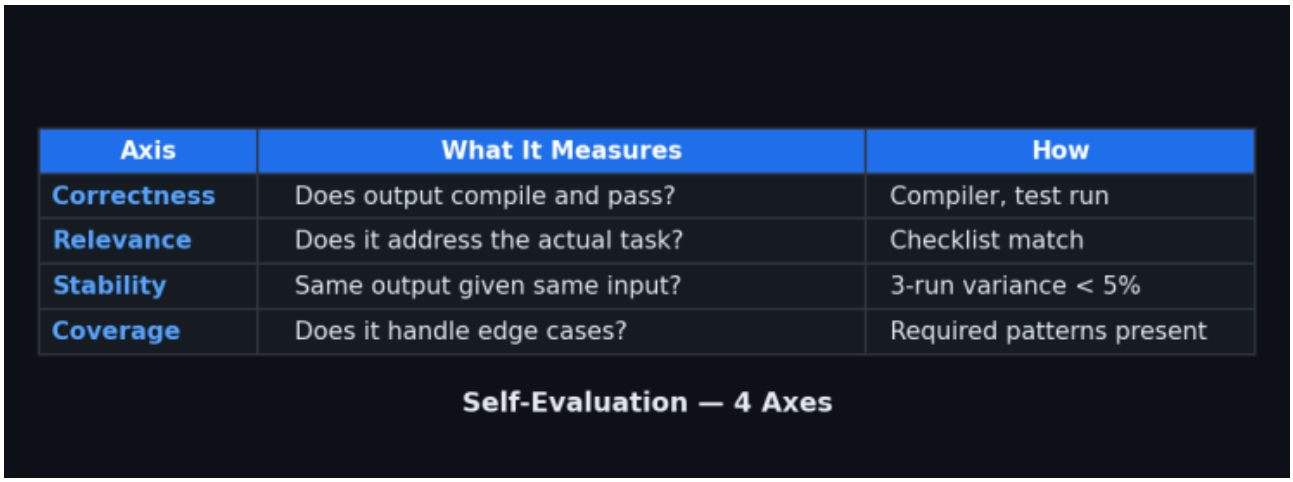
⚡ Layer 2: Skill + Self-Evaluation

The skill layer has domain knowledge (SKILL.md) plus something Gulin's model doesn't have: **embedded self-evaluation across 4 axes.**

Minimize image

Edit image

Delete image



Axis	What It Measures	How
Correctness	Does output compile and pass?	Compiler, test run
Relevance	Does it address the actual task?	Checklist match
Stability	Same output given same input?	3-run variance < 5%
Coverage	Does it handle edge cases?	Required patterns present

Self-Evaluation — 4 Axes

Each skill evaluates itself after generation. No external LLM. No human judge. The 4 axes are designed to catch the 4 ways skills fail in practice.

🔗 Layer 3: BDD Harness

This is the layer that makes everything executable.

AI researcher **Rinat Abdullin** mades the point that started this whole thread: BDD with Given-When-Then is the natural validation layer for AI-generated code. SDD (Spec-Driven Development) produces dead specs. BDD produces **AI-executable specs**.

Minimize image

Edit image

Delete image

If the code violates the spec → build fails. Not "let's schedule an audit." Not "the document might be stale." The pipeline stops.

An agent can run this harness 100 times in a single session. Each run verifies that the output matches the spec. No drift. No stale documents. No confident garbage.

🏗️ From Code Generation to Behavior Design

This is the fundamental difference — and it came from an unexpected place.

One man who built Magnum Studio (Jira → AI agent pipeline), put it this way:

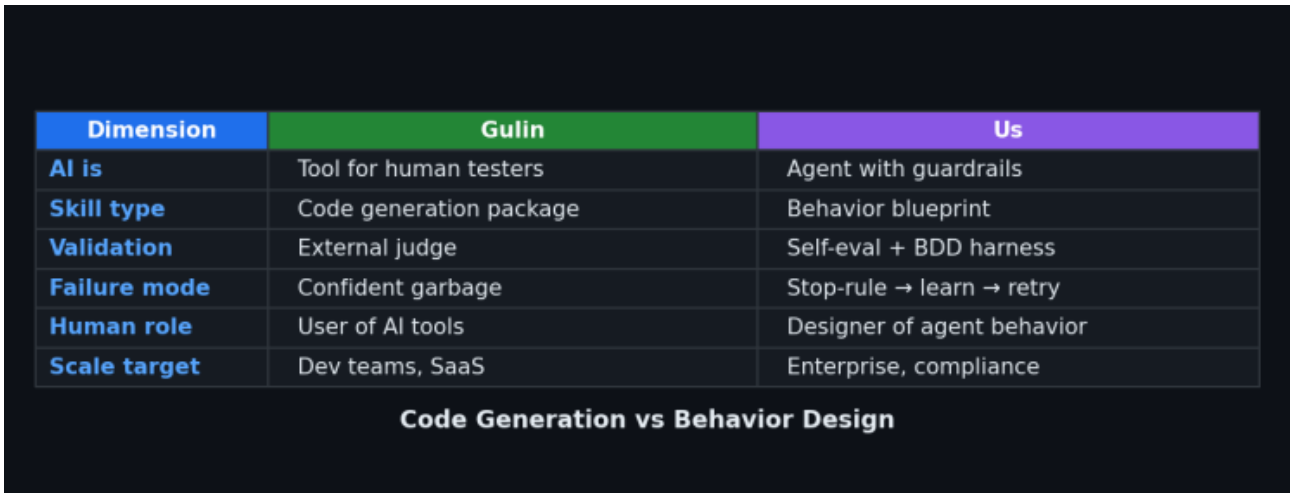
"QA thinking is about designing agent behavior. It's the same muscle, but on the other side."

He's right. Gulin treats skills as **code generation packages**: load the skill, generate a test, move on. We treat skills as **agent behavior blueprints**: load the skill, and the agent knows not just what to write, but what decisions to make, when to ask for help, and how to fail honestly.

Minimize image

Edit image

Delete image



Dimension	Gulin	Us
AI is	Tool for human testers	Agent with guardrails
Skill type	Code generation package	Behavior blueprint
Validation	External judge	Self-eval + BDD harness
Failure mode	Confident garbage	Stop-rule → learn → retry
Human role	User of AI tools	Designer of agent behavior
Scale target	Dev teams, SaaS	Enterprise, compliance

Code Generation vs Behavior Design

📊 The Numbers

8 skills. 4 self-evaluation axes. 9 stop-rules. All running on a single model with no external judge.

My previous article showed 0% lift on **correctness** — skills don't help the model write more correct code on patterns it already knows. But that was the wrong metric. Skills help with **consistency** — same patterns every time, vocabulary, hygiene. And they help with **behavior** — what the agent does when it's uncertain, when it discovers a conflict, when it needs to fail honestly.

Gulin's skill-creator is at ★123 and growing. His approach works for teams with an external evaluation budget. For teams that need self-validating skills with enterprise guardrails — ours works.

Both approaches are valid. They solve different problems.

💬 What I'd Tell Anton

Your skill-creator is the best thing that happened to OpenCode skills. Your 3-Layer Architecture should be required reading. Your Apple story is earned, not branded.

But skills are not npm packages. They're behavior blueprints with context, self-evaluation, and executable validation.

The next evolution isn't better prompts. It's better architecture.

Which model fits your team — tools that augment human testers, or agents with built-in guardrails? Try both. The difference shows up in the second week.

References

[1] **Tom Piaggio, "Mutation Testing vs Code Coverage: The Real Test-Quality Metric"**, Autonoma Blog, June 2026

Victor Ematin · AI Quality Engineering Lead · OpenCode Go

#TestAutomation #AIQA #AgenticTesting #Playwright #OpenCode